

Ahsaullah University of Science and Technology
Department of Computer Science and Engineering

Course Code : CSE 3200

Student Resource Hub

A Centralized Platform for Academic Resource Sharing

Aligned with UN Sustainable Development Goals 4 & 10

Built on the ASP.NET Core MVC (C#) Framework

Project Proposal Report

Lab Section : B1

Team Members:

Rifah Tasnia Mahek - 20230104067

Dibbya Stutrom Dey - 20220204017

Abdullah Al Rabbie - 20230104057

Table of Contents

- 1. Project Overview
 - 1.1 Objective.....
 - 1.2 Target Audience.....
- 2. Problem Statement.....
 - 2.1 SDG 4 — Quality Education (Primary).....
 - 2.2 SDG 10 — Reduced Inequalities (Secondary).....
- 3. Solution Approach
- 4. Key Features
 - 4.1 Core Features
 - 4.2 SDG 10 — Equity-Focused Additions.....
 - 4.3 User Roles.....
 - 4.4 CRUD Operations.....
- 5. Technology Stack
 - 5.1 Key Application Routes / Controller Actions
- 6. System Design
 - 6.1 System Architecture Diagram.....
 - 6.2 Use Case Diagram
 - 6.3 Entity-Relationship (ER) Diagram.....
- 7. Project Milestones
- 8. Background Study & Gap Analysis

1. Project Overview

1.1 Objective

The Student Resource Hub is a student-friendly web application designed to centralize access to academic resources — previous years' question papers, lecture slides, class notes, and study materials — reducing dependency on scattered sources such as chat groups and shared folders. The application is built end-to-end on the ASP.NET Core MVC framework using C#.

1.2 Target Audience

- University students who need quick, reliable access to past questions and notes
- First-year students who struggle to locate reliable resources when they join a new department
- Class representatives and seniors who want an efficient way to share materials with juniors

2. Problem Statement

Academic resources at most universities are shared informally — scattered across Facebook groups, personal chat threads, and ad-hoc shared drives. This creates several persistent problems:

- Resources are hard to discover: students must scroll through chat history or ask around to find past question papers or notes.
- First-year and transfer students face a steep access barrier since they are often outside the informal networks (senior group chats, batch pages) where resources circulate.
- There is no quality control: duplicate, outdated, or irrelevant files clutter shared spaces, and there is no way to verify a resource before relying on it.
- Access depends on individual goodwill rather than a sustainable, structured system — when a contributor becomes inactive, that channel of resources dries up.

These issues map directly onto two United Nations Sustainable Development Goals that this project targets:

2.1 SDG 4 — Quality Education (Primary)

By centralizing academic resources into a single, well-organized, searchable platform, the project directly supports inclusive and equitable access to learning materials — a core target of SDG 4.

2.2 SDG 10 — Reduced Inequalities (Secondary)

The platform reduces the access gap between students who are well-connected to informal resource-sharing networks and those who are not — particularly first-year students and newcomers — by making resources openly available on one platform rather than gated behind social connections.

3. Solution Approach

The Student Resource Hub addresses these problems through a role-based ASP.NET Core MVC web application where students can browse and download categorized academic resources, and class representatives or admins can upload and curate content through server-rendered Razor views. The system is built around three pillars:

- Centralization — one platform for all subjects and semesters, replacing scattered chat-based sharing.
- Curation — an approval workflow, enforced through role-based authorization in the Controllers, ensures only relevant, verified resources are published.
- Accessibility — an open, no-paywall model with a guided onboarding flow for first-year students, directly supporting the SDG 10 angle.

4. Key Features

4.1 Core Features

- User authentication with role-based access (Student, Admin/CR) via ASP.NET Core Identity
- Resource access: view and download materials filtered by subject and semester
- Resource upload for CRs/Admins, supporting PDF, DOCX, and PPT formats
- Search and filter across subjects, semesters, and resource types

4.2 SDG 10 — Equity-Focused Additions

- Free, no-paywall access model for all registered students
- First-year "starter kit" onboarding: a guided landing flow that filters resources by department and semester for new students
- Offline packs: downloadable ZIP bundles of a semester's resources for low-connectivity access
- Low-data mode (stretch goal / future work) for students on limited data plans

4.3 User Roles

- Students — view/download resources, search and filter
- Admin/CR — approve or remove resources, manage subjects and users

4.4 CRUD Operations

The system supports full CRUD (Create, Read, Update, Delete) operations across three core entities: Users, Resources, and Subjects, implemented through ASP.NET Core Controllers and Entity Framework Core.

5. Technology Stack

- Backend & Frontend: ASP.NET Core MVC (C#) with Razor Views — a single, unified framework for both presentation and application logic
- UI Styling: Bootstrap (bundled with the default ASP.NET Core MVC template)
- Database: SQL Server — stores users, subjects, resources, uploads, and metadata
- ORM / Data Access: Entity Framework Core (Code-First migrations)
- Authentication & Authorization: ASP.NET Core Identity with cookie-based authentication and role-based access control (Student / Admin)
- Supporting Tools: Git & GitHub for version control; Visual Studio for development

5.1 Key Application Routes / Controller Actions

HTTP Method	Route	Controller / Action	Description
GET	/Account/Register	AccountController.Register (GET)	Display the registration form
POST	/Account/Register	AccountController.Register (POST)	Create a new user account via ASP.NET Core Identity
GET	/Account/Login	AccountController.Login (GET)	Display the login form
POST	/Account/Login	AccountController.Login (POST)	Authenticate the user and start a cookie session
GET	/Resources/Index	ResourcesController.Index	List/search resources (filter by subject, semester)
GET	/Resources/Details/{id}	ResourcesController.Details	View a single resource

HTTP Method	Route	Controller / Action	Description
GET, POST	/Resources/Create	ResourcesController.Create	Upload a new resource (CR/Admin only)
POST	/Resources/Delete/{id}	ResourcesController.Delete	Remove a resource (Admin only)
GET	/Subjects/Index	SubjectsController.Index	List all subjects and semesters

6. System Design

6.1 System Architecture Diagram

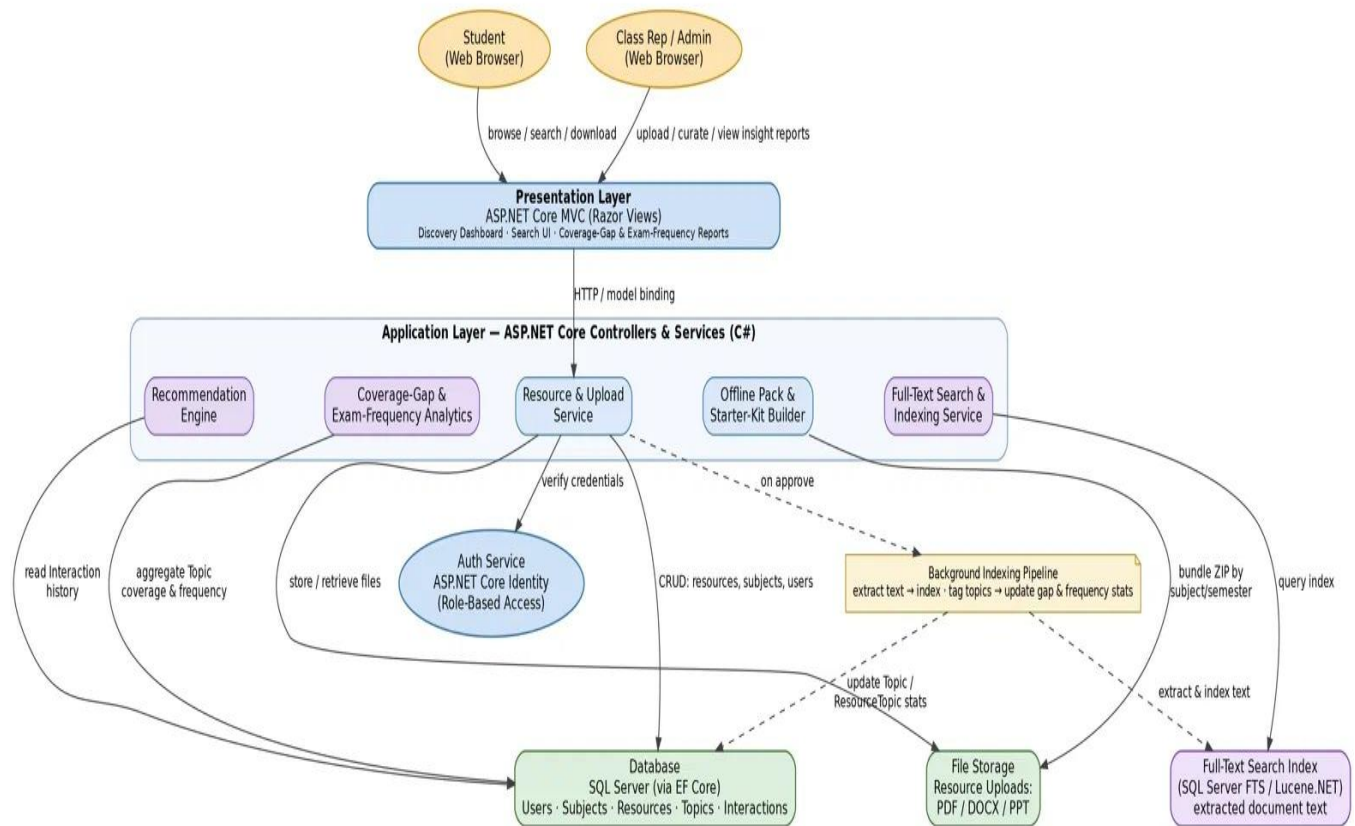


Figure 1: High-level system architecture of the Student Resource Hub (ASP.NET Core MVC)

6.2 Use Case Diagram

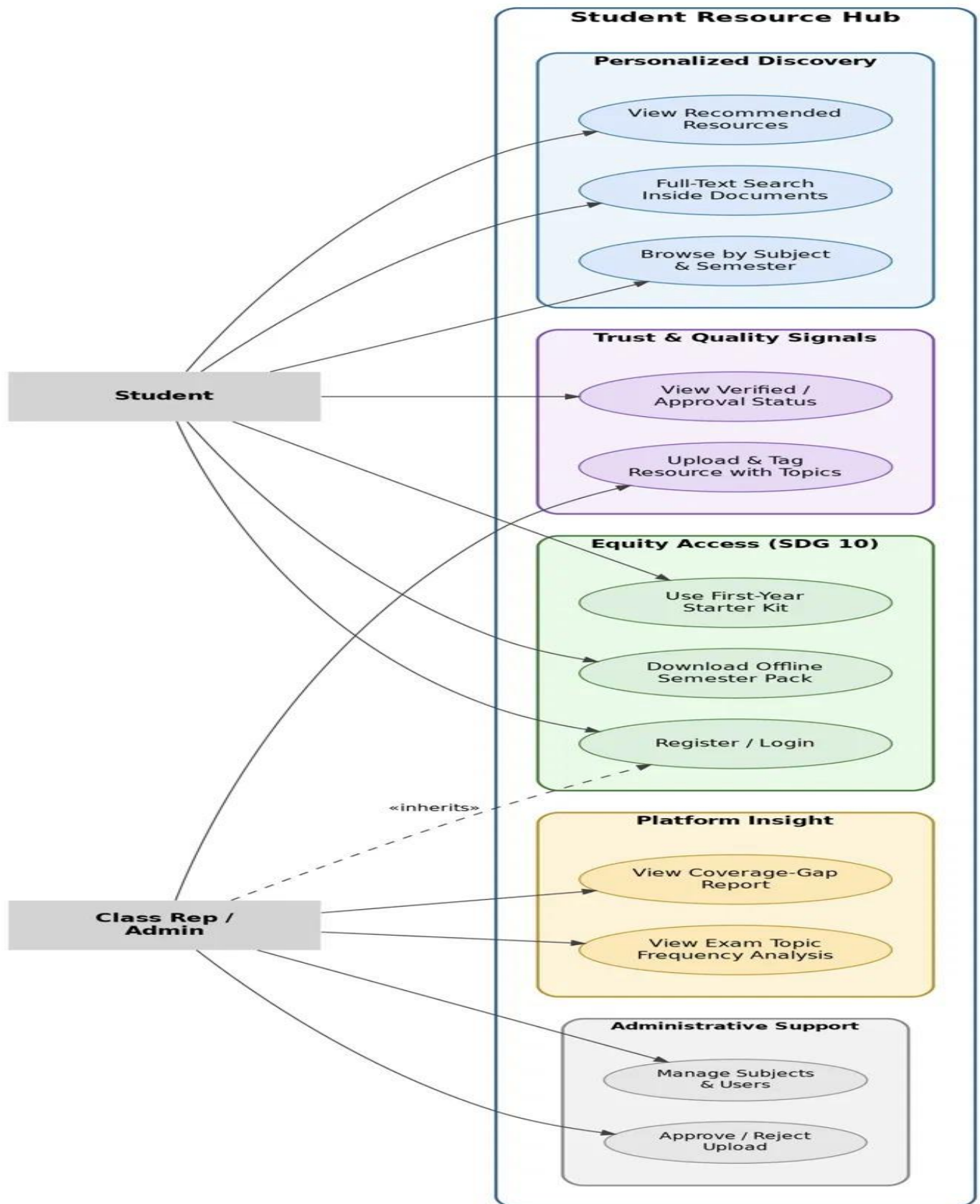


Figure 2: Use case diagram showing Student and Admin/CR interactions

6.3 Entity-Relationship (ER) Diagram

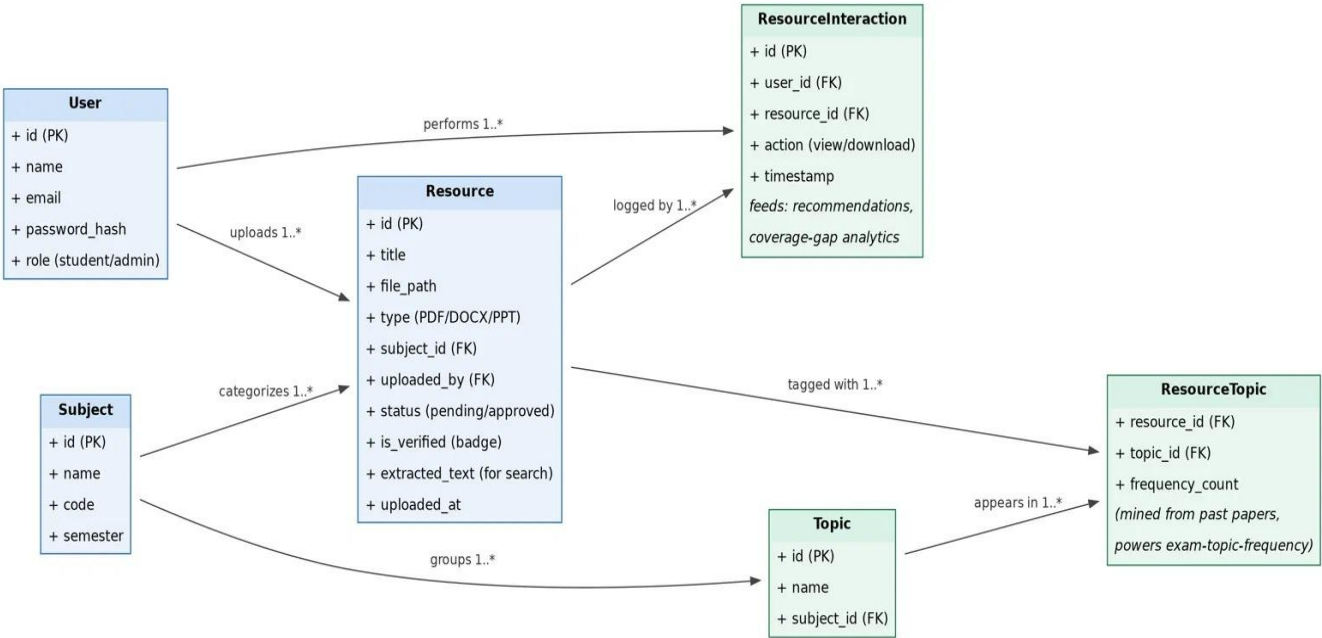


Figure 3: Entity-relationship diagram for User, Subject, and Resource entities

7. Project Milestones

Milestone	Key Deliverables
1. Core Setup & Authentication	Database schema design with EF Core migrations, ASP.NET Core Identity setup with role-based authentication (cookie-based), registration/login Razor views, basic MVC scaffolding and Bootstrap layout
2. Resource Management	Upload & storage pipeline (wwwroot / file storage), subject/semester categorization, search & filter functionality via Controllers and LINQ queries
3. Finalization & Testing	Admin approval workflow, UI polish, unit/integration testing, deployment preparation (IIS / Azure App Service)

8. Background Study & Gap Analysis

Most students currently rely on informal channels — shared Google Drive folders, Facebook groups, and batch chat threads — to exchange academic resources. While functional at a small scale, these approaches lack structure, moderation, and long-term reliability. The table below compares these existing practices against the proposed system.

Aspect	Existing Systems (e.g., shared Drive folders, Facebook groups)	Student Resource Hub
Organization	Unstructured, scattered across chats/folders	Structured by subject & semester with search/filter
Access control	No role separation; anyone can post anything	Role-based access (Student / Admin) via ASP.NET Core Identity, with approval flow
Discoverability	Manual scrolling, no search	Search & filter by subject, semester, resource type

Aspect	Existing Systems (e.g., shared Drive folders, Facebook groups)	Student Resource Hub
Quality control	No moderation, duplicate/irrelevant files	Admin/CR approval before publishing
Accessibility for new students	Hard to find as an outsider to group chats	Centralized, open platform easing first-year onboarding
Sustainability	Depends on individual sharing habits	Persistent, structured repository independent of any single contributor

This gap analysis reinforces why the platform's SDG 4 and SDG 10 framing is substantive rather than superficial: the core features (centralized structure, role-based moderation implemented via ASP.NET Core Identity, and equitable free access) directly resolve the shortcomings observed in current informal practices, rather than simply digitizing them.